Asynchronous Processing

IF3110 – Web-based Application Development
Week 11 · Event-driven asynchronous processing

Beyond HTTP waiting with background jobs, queues, and events

The 30-second report request



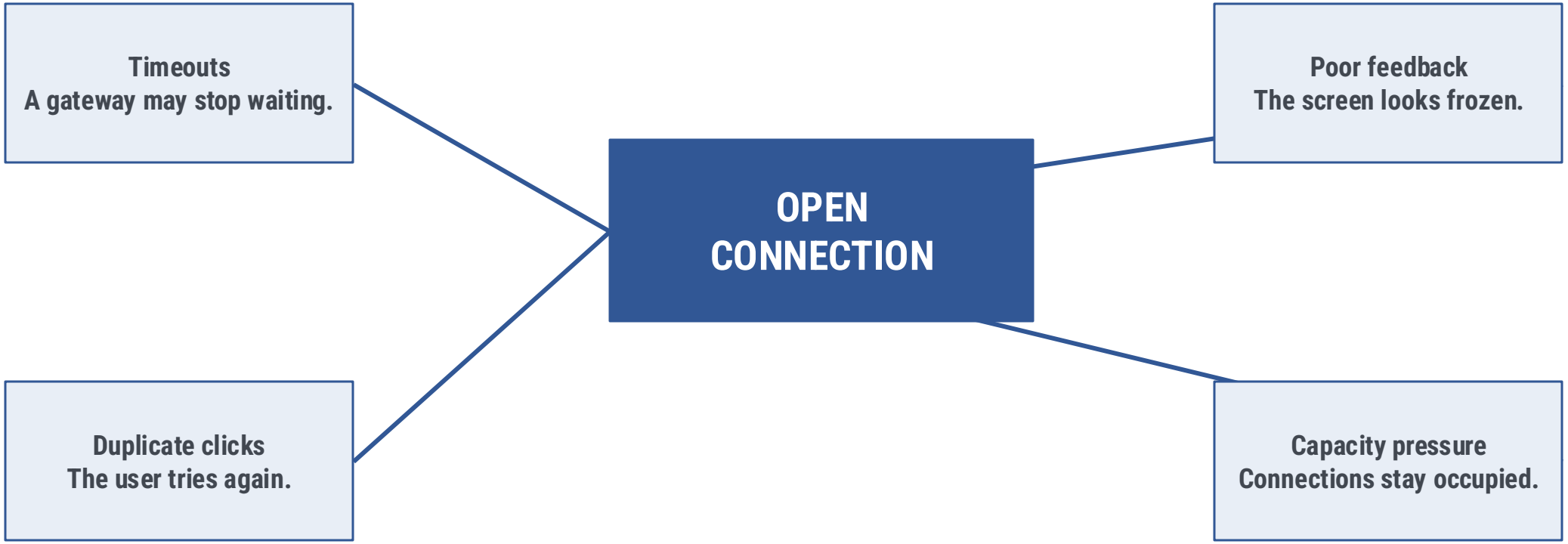
What can fail before the response arrives?

timeout

retry click

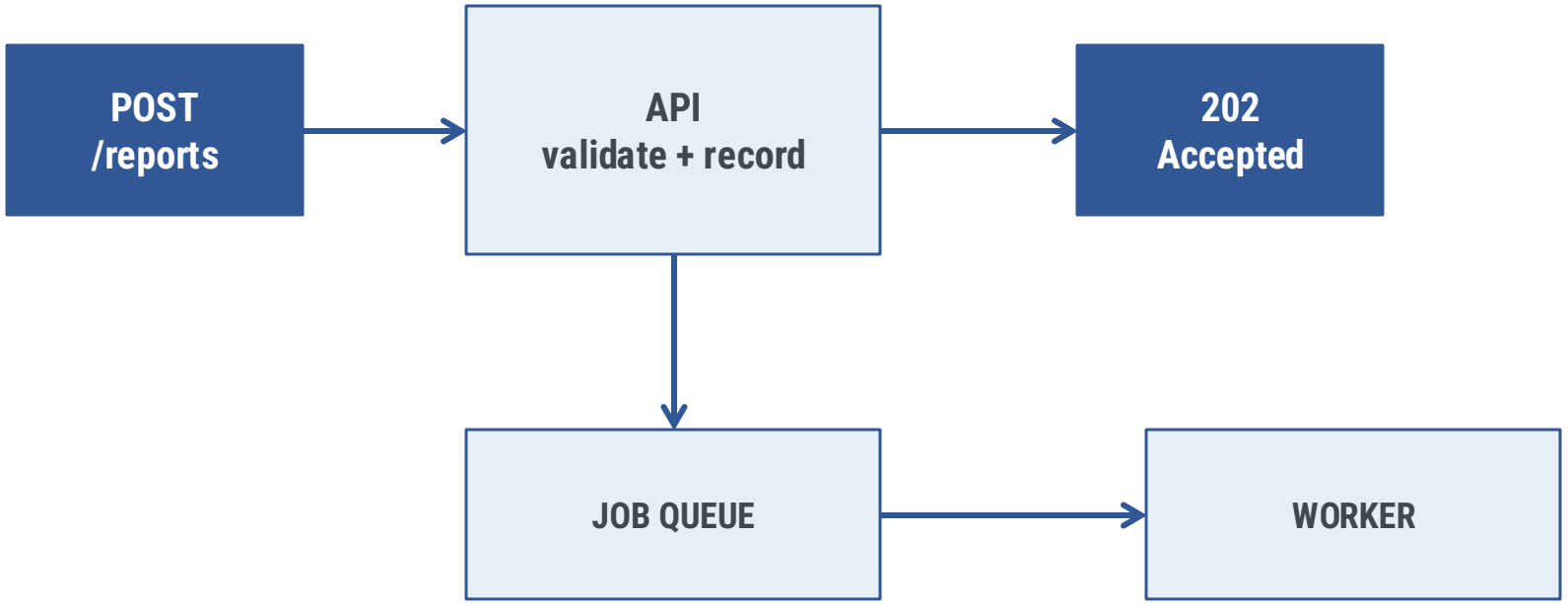
lost result

When HTTP waits



Long work and long requests are different problems

Accept now, finish later



The response ends.
The work continues.

Client receives a job ID
and checks status later.

Two kinds of asynchronous work

JavaScript async / await

One process

await pauses this function
while the event loop can do other work.

The caller may still be waiting.

Distributed async processing

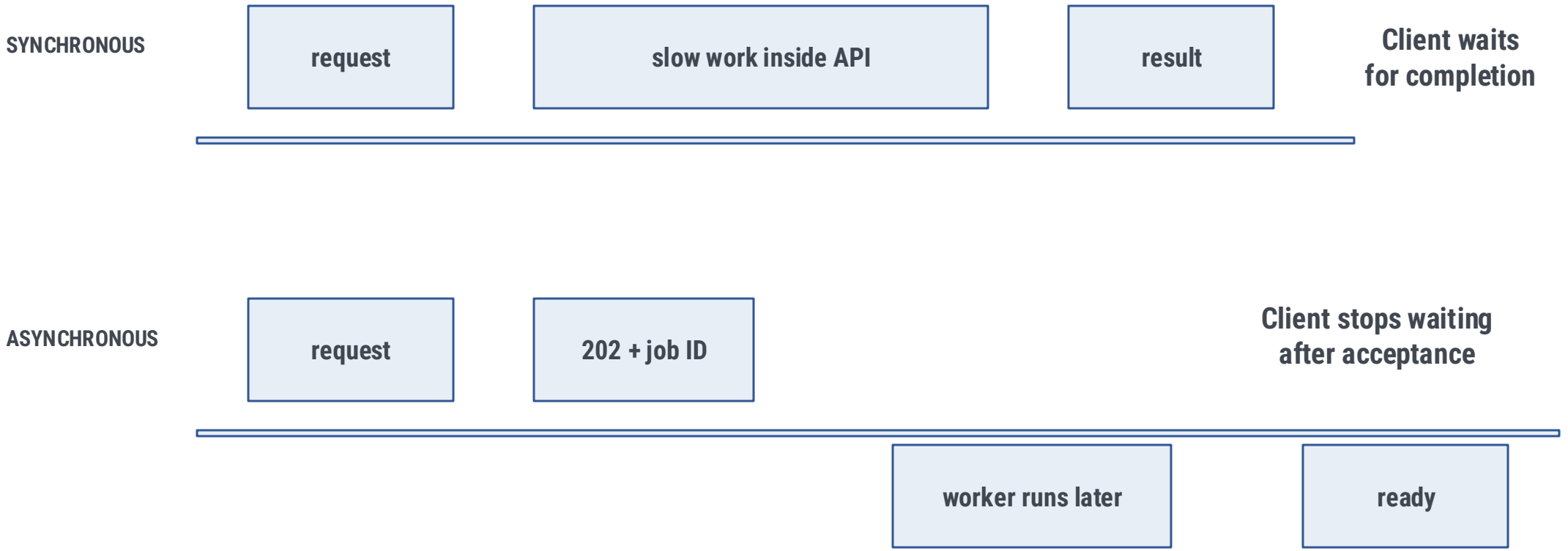
Multiple processes

The API records work and returns.
A worker continues later.

A broker carries the message.

Same word. Different system boundary.

Request timeline



Acceptance becomes a separate moment from completion

Foreground and background work

FOREGROUND

- Authenticate the caller
- Validate input
- Store the job request
- Return job ID and status URL

BACKGROUND

- Query large datasets
- Generate the file
- Upload the artifact
- Send the download link

Boundary test: what must finish before the API can say “accepted”?

HTTP 202 Accepted

202 means accepted

It does not mean the work succeeded.

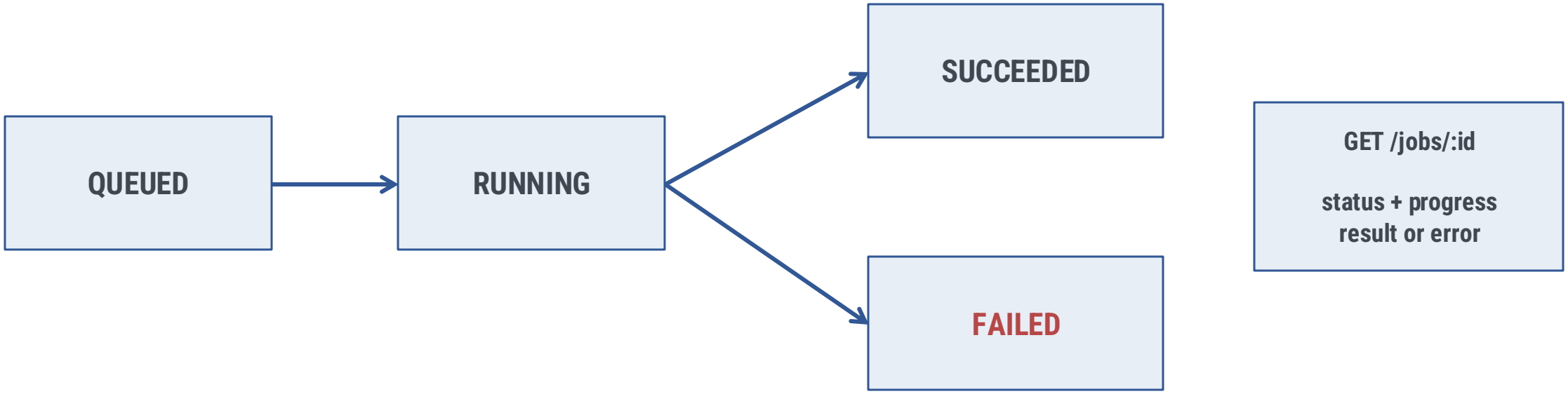
- return an identifier
- expose status
- define failures

202 Accepted

```
{
  "jobId": "rpt_123",
  "status": "queued",
  "statusUrl": "/jobs/rpt_123"
}
```

Client can poll, receive a webhook, or follow a live update.

Job status lifecycle



A job resource makes asynchronous work visible

02

Event-driven architecture

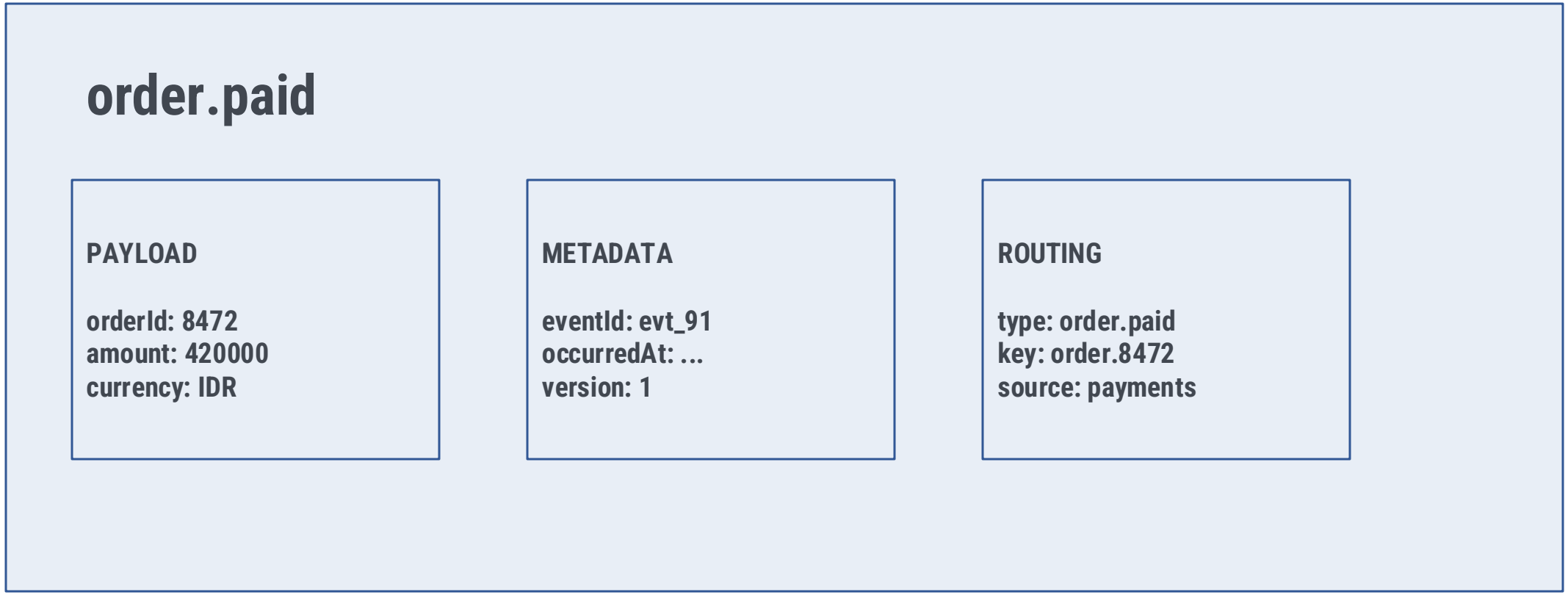
Components react to facts and requests carried through a broker.

Producer, broker, consumer



The producer does not need to know when the consumer runs.

Event anatomy



A useful event says what happened and carries enough context.

Event schema and versioning

order.paid · v1
orderId: 8472
amount: 420000
currency: IDR
version: 1

evolve

order.paid · v2
orderId: 8472
amount: 420000
currency: IDR
couponCode: PROMO10 new, optional
version: 2

COMPATIBLE
- Add an optional field
- Consumers ignore unknown fields

BREAKING: publish a new version
- Rename or remove a field
- Change a field's type or meaning

The event schema is a contract between teams, not an internal detail.

Commands and events

COMMAND

SendReceipt

Intent
Imperative name
Usually one intended handler

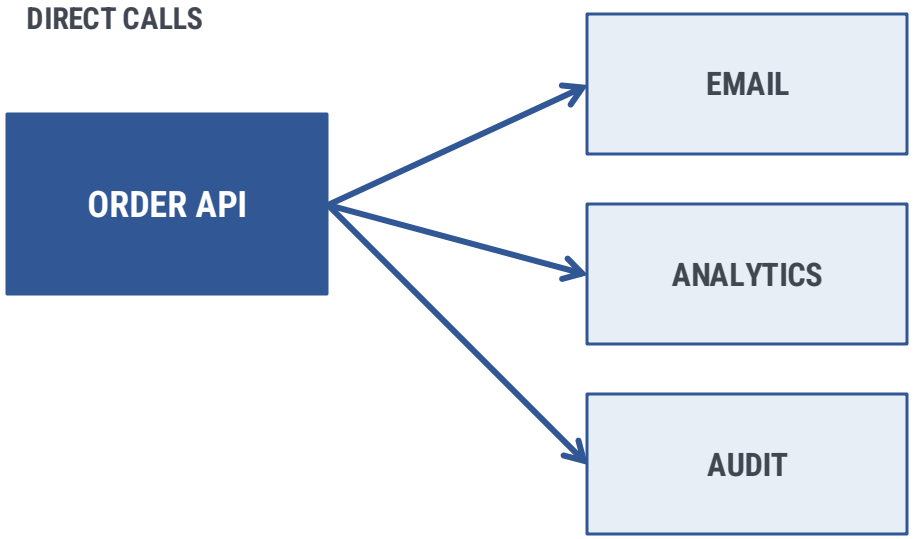
EVENT

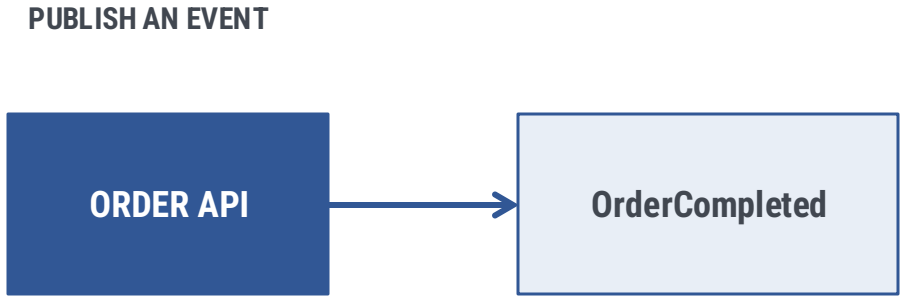
OrderPaid

Fact
Past-tense name
Zero, one, or many consumers

Naming reveals ownership: “please do” versus “this happened”

Loose coupling





New consumers can subscribe without changing the order API.

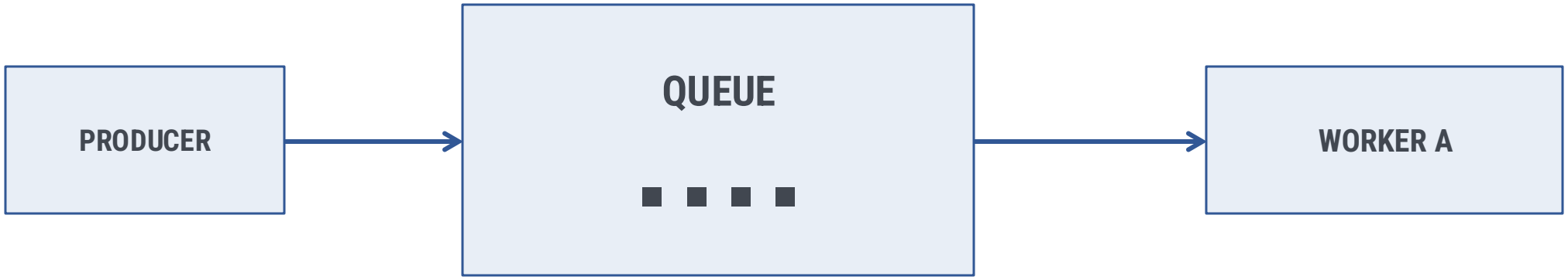
Loose coupling moves coordination from direct calls to message contracts.

The broker



A broker creates time between producer and consumer.

Queue: one job, one worker

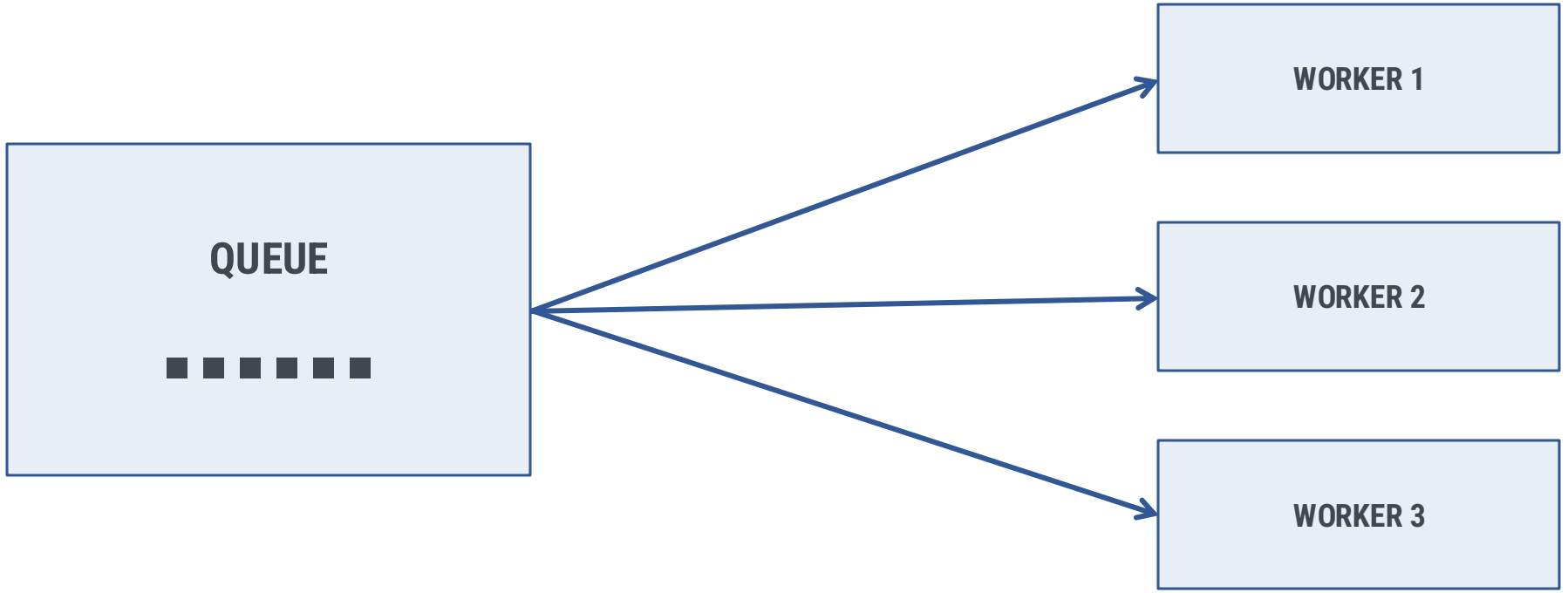


Competing consumers share work.
Each message goes to one available worker.

report generation

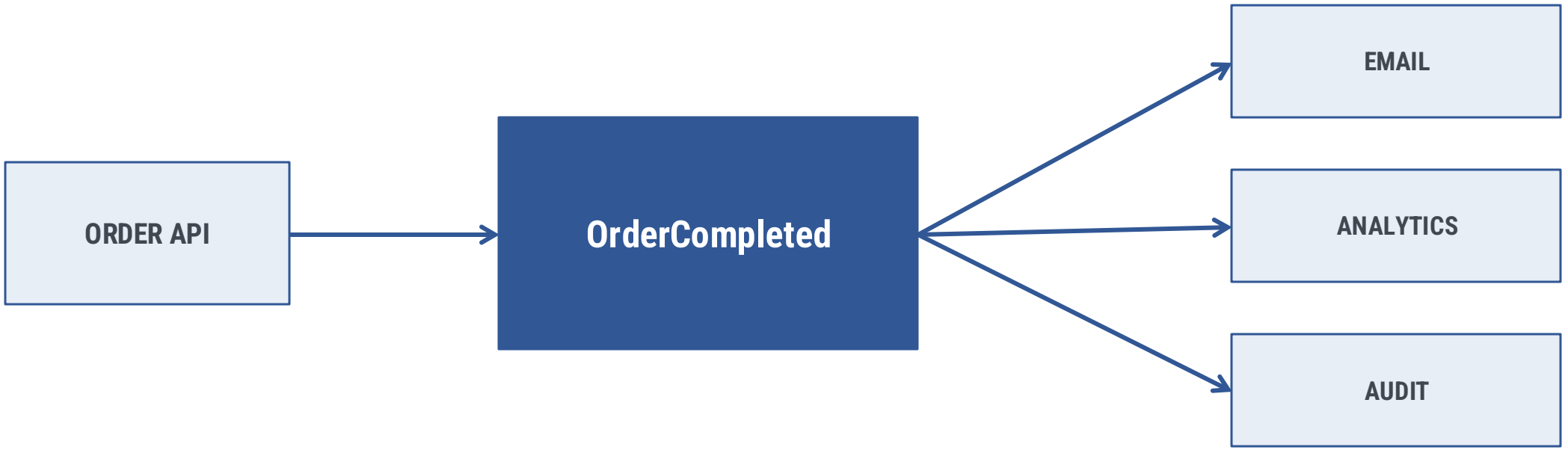
image resizing

More workers, same queue



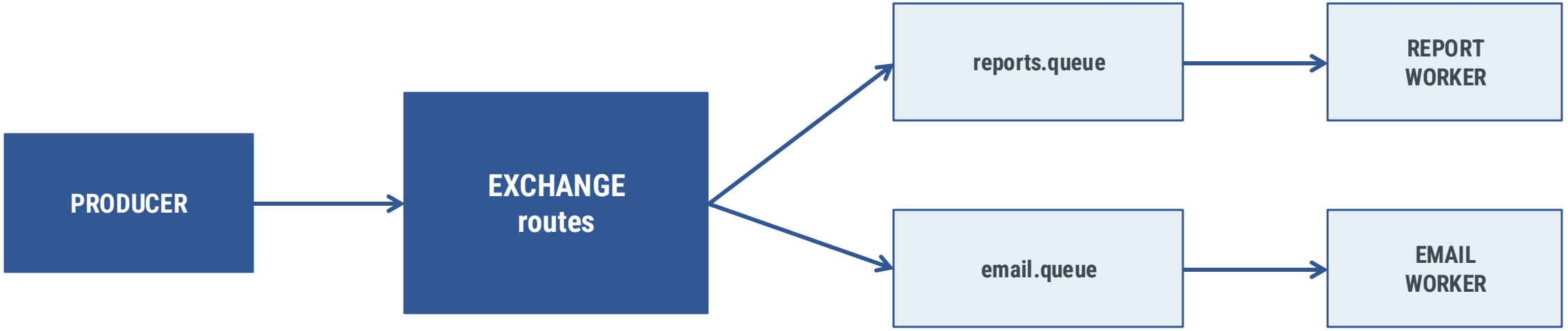
Throughput can grow with worker count.
Ordering may become harder.

Pub/sub: one event, many consumers



Each subscription receives its own copy and progresses independently.

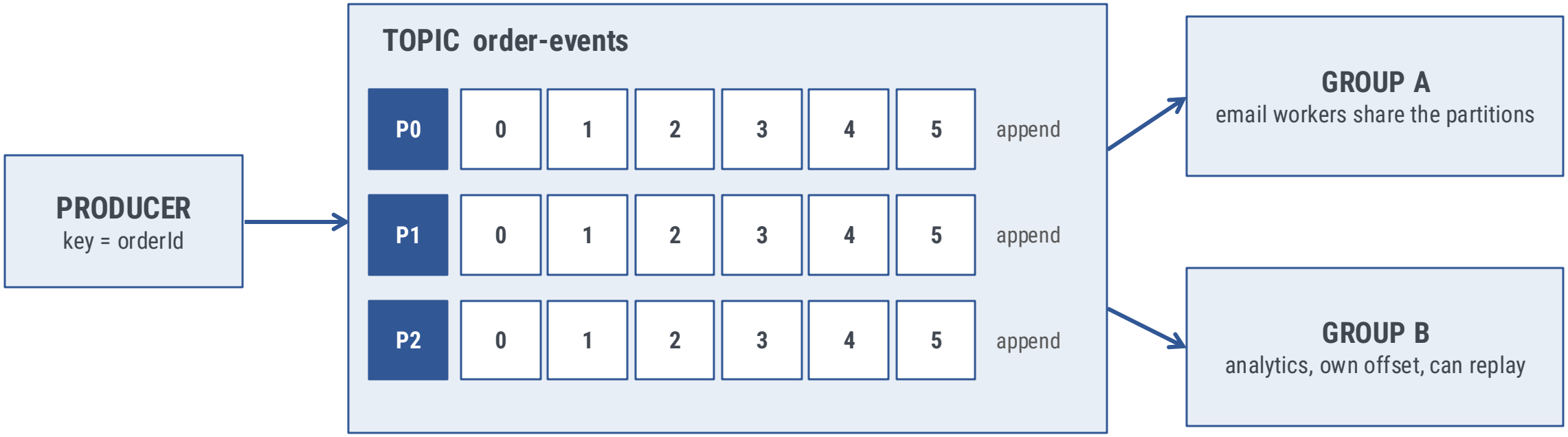
RabbitMQ conceptual map



routing key + binding decide the path

Conceptual goal: understand the roles before configuration.

Log-based broker: Kafka and Redpanda



The log keeps messages. Each consumer group tracks its own offset.

Order holds within a partition: same key, same partition. Redpanda speaks the Kafka API.

Queue broker or log broker?

	Queue broker (RabbitMQ)	Log broker (Kafka / Redpanda)
After a message is handled	Removed once acknowledged	Stays in the log until retention expires
Replay old events	Not by default	Yes, reset the consumer offset
Ordering	Per queue; weaker with competing workers	Per partition (same key, same partition)
Scaling consumers	Add workers to one queue	Up to one consumer per partition in a group
Typical use	Task queues, routing rules	Event streams, audit, analytics, many readers

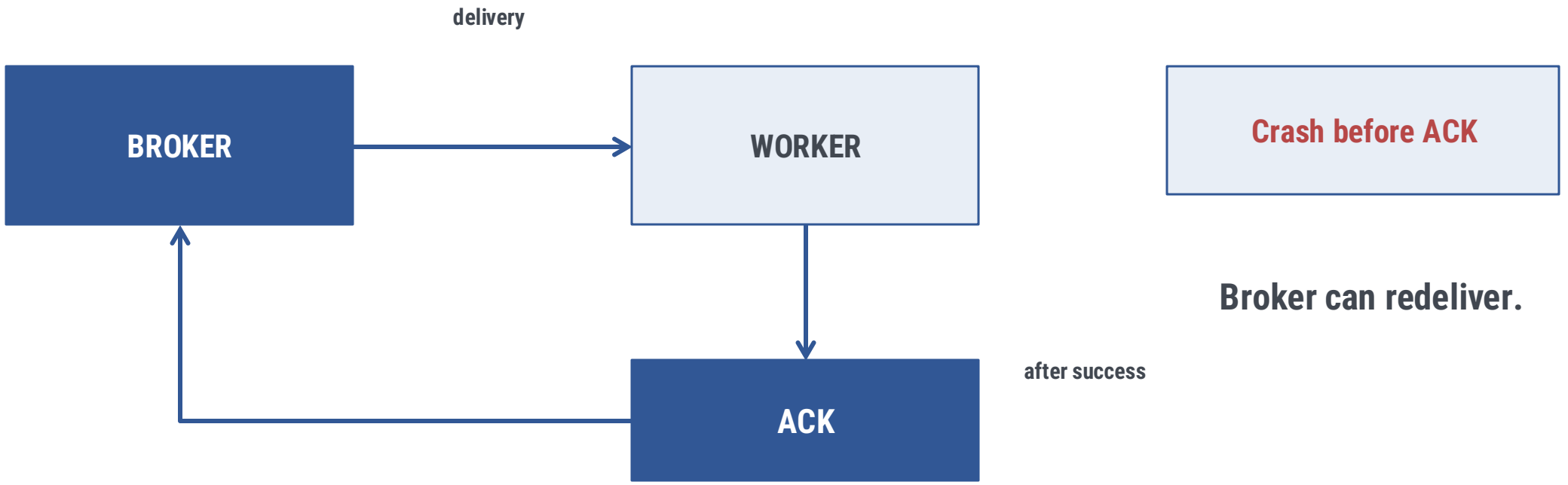
Choose by what should happen to a message after it is handled.

03

Reliable message processing

Delivery crosses process and network boundaries. Design for partial failure.

Acknowledgement



Delivery and successful processing are separate moments.

Retry and backoff



Backoff gives a failing dependency time to recover.
A retry limit prevents endless loops.

transient error

max attempts

add jitter

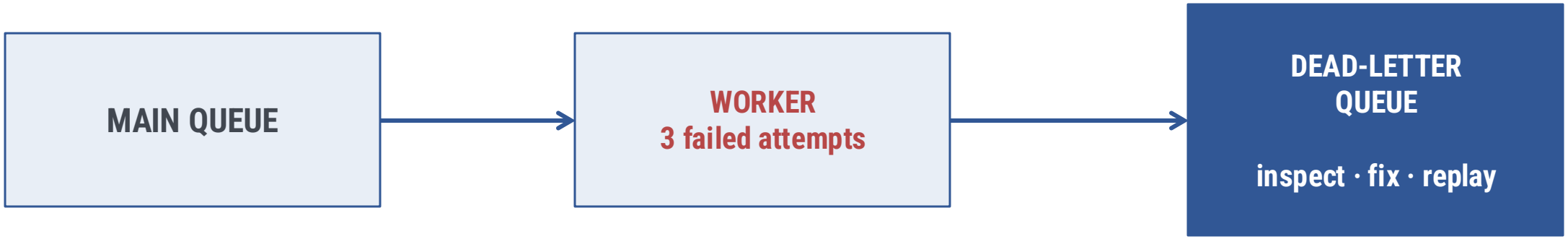
Duplicate delivery and idempotency



Idempotent handling makes repeated delivery safe.

Use a stable message or business key and store the outcome.

Dead-letter queue



A DLQ isolates poison messages so healthy work can continue.

observe

repair

replay

Eventual consistency

TIME 0

ORDER API

status: paid

TIME + 2 s

EMAIL SERVICE

receipt: pending

TIME + 5 s

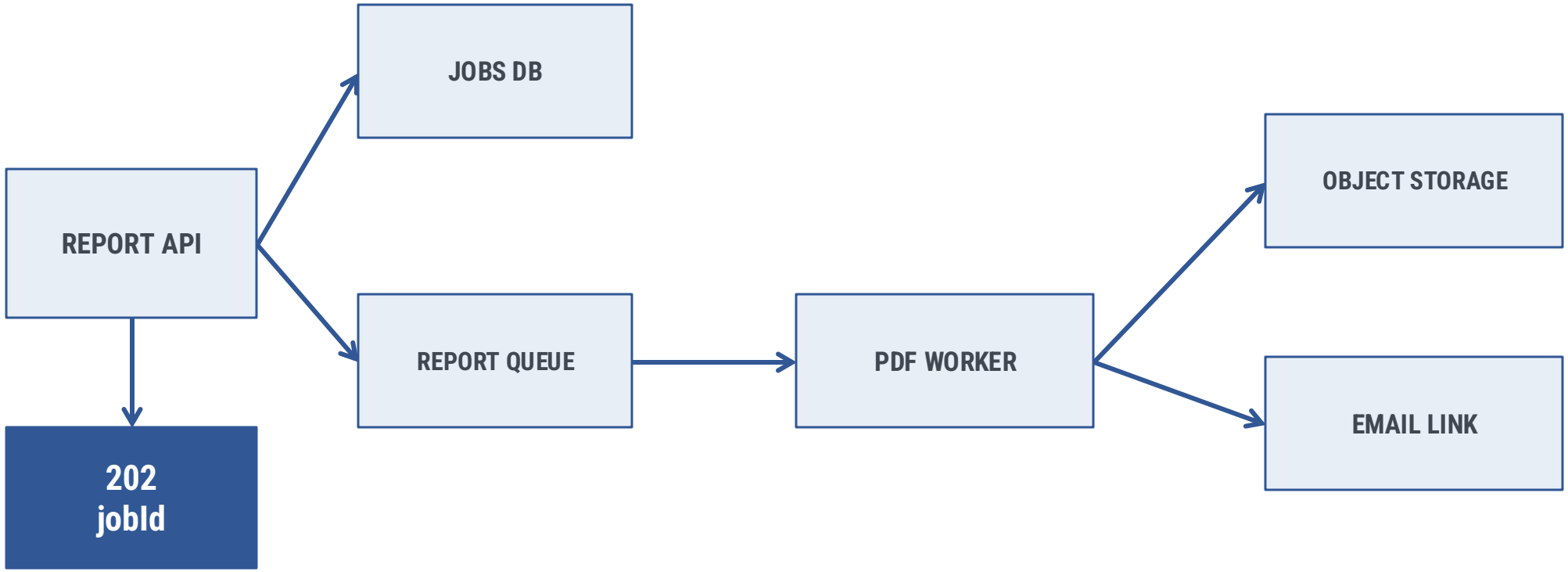
CUSTOMER VIEW

receipt: sent

Different components can be correct at different moments.

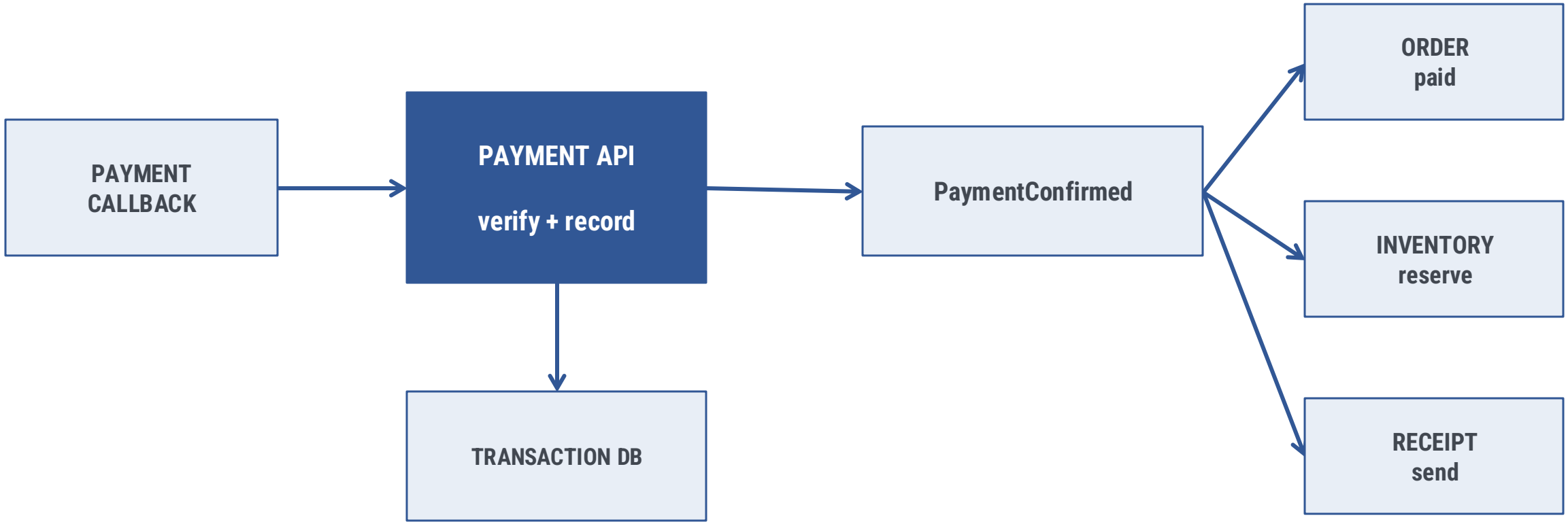
The UI should explain “processing” instead of pretending everything is instant.

Case study: report generation



Discussion: where are status, retries, idempotency, and the final URL stored?

Case study: payment and order processing



Critical question: what must commit before the event is published?

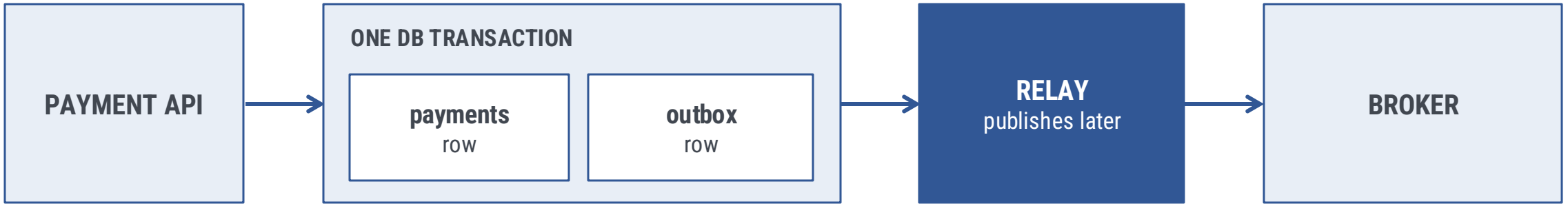
Transactional outbox

DUAL WRITE: RISKY



Crash between the two steps: paid order, no event.

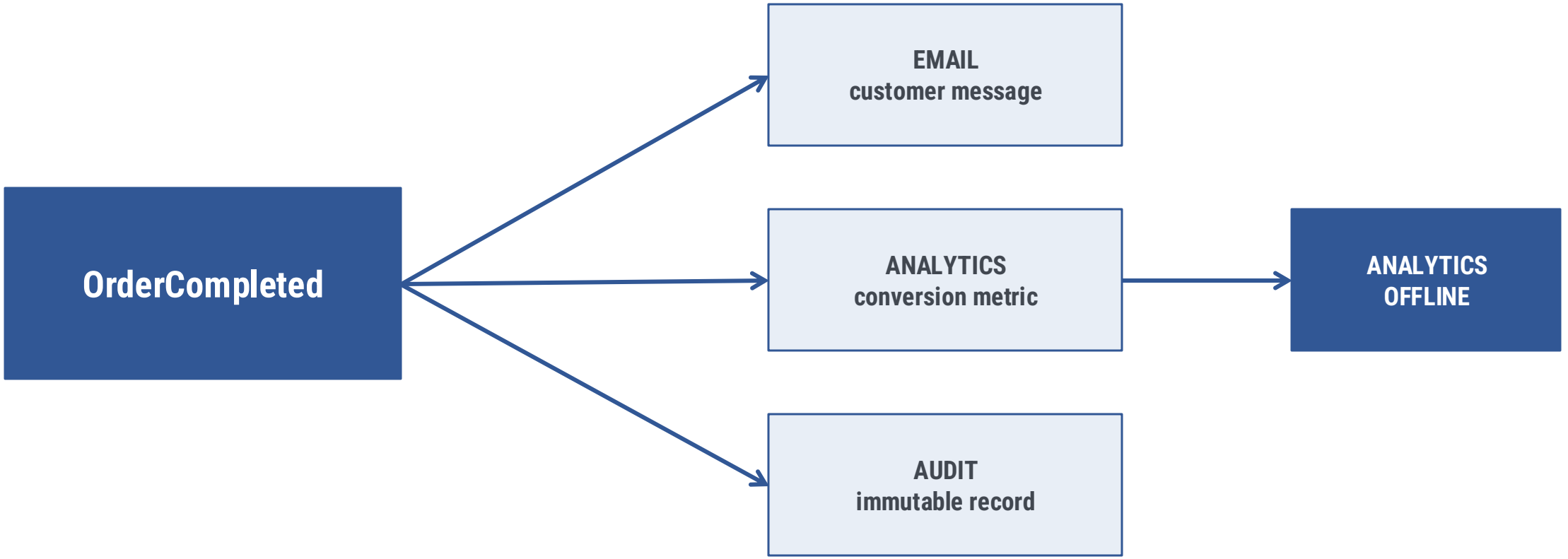
OUTBOX: SAFE



Commit the fact and the event together. Publish afterward.

The relay may publish twice, so consumers must stay idempotent.

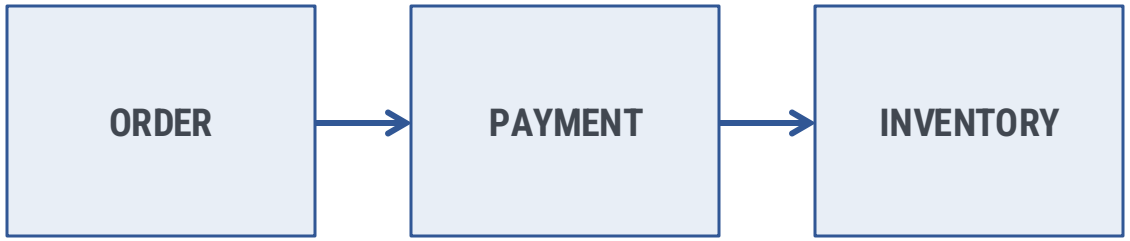
Case study: independent consumers



Email and audit continue while analytics recovers and catches up.

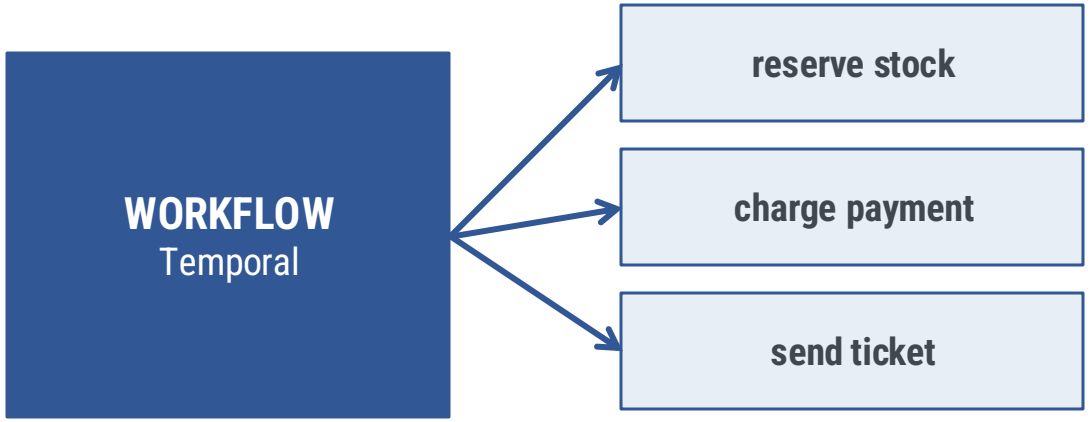
Many steps: events or a workflow?

CHOREOGRAPHY: events only



Each service reacts to the previous event. Nobody owns the whole flow.

ORCHESTRATION: workflow engine



The engine stores progress, retries steps, runs timers, and undoes earlier steps on failure.

Use a workflow for many steps, long waits, or undo. Use pub/sub for simple fan-out.

Decision framework

Mechanism	Best when	Client experience	Example
HTTP request–response	Result is needed now	Waits for result	Login validation
Background job	One app owns slow work	202 + status	PDF report
Queue	One task needs one worker	Result later	Image resize
Pub/sub	Many services react	No single response	Order completed
WebSocket / SSE	Server pushes updates	Connection stays open	Progress or live feed
Workflow engine	Many steps, waits, or undo	Status or result later	Ticket purchase, booking

Choose by response timing, ownership, fan-out, and update direction.

Architecture challenge

Campus event ticket purchase

A student pays, receives a ticket, updates analytics, writes an audit record, and sees remaining capacity change live.

IN PAIRS · 6 MIN

1. **Mark synchronous steps**
2. **Choose queue or pub/sub**
3. **Add status or live updates**
4. **Mark one idempotency key**

ACCEPT EARLY **DECOUPLE WITH MESSAGES** **DESIGN FOR RETRIES**

Exit ticket: name one step that must be idempotent and explain why.